
Plantilla de Programación Competitiva - Python

Universidad de Bogotá Jorge Tadeo Lozano

Facultad de Ciencias Naturales e Ingeniería

Semillero de Programación Competitiva

Maratón Nacional ACIS REDIS 2026

Elaborado por: Nicolás Leitón

Bogotá D.C., 14 de julio de 2026

Índice

1. Leer Input	1
1.1. n inicial y luego casos	1
1.2. Terminar cuando es 0 0	1
1.3. Enumerar casos hasta EOF	1
1.4. Input matriz nxm hasta EOF	2
2. Output Format	2
3. Macros	3
4. Estructuras de Datos	3
4.1. Collections - Deque	3
4.2. Heapq - Priority Queue	3
4.3. DefaultDict - Grafos	4
4.4. Counter - Frecuencias	4
4.5. Bisect - Búsqueda Binaria	5
4.6. Sets - Conjuntos y operaciones en $O(1)$	5
4.7. Matrices	6
5. Algoritmos	7
5.1. Two Sum - Doble puntero	7
5.2. Sliding Window - Suma Mínima	8
6. Grafos	8
6.1. Dijkstra - Camino menos costoso	8
6.2. BFS - Camino más corto	9
6.3. BFS - Para obtener distancias/profundidad	10
6.4. DFS - Recursivo	11
6.5. DFS - Iterativo	11

1. Leer Input

1.1. n inicial y luego casos

```
1 import sys
2 inputs = iter(sys.stdin.readlines())
3 TC = int (next(inputs))
4 for _ in range(TC):
5     print(sum(map(int, next(inputs).split())))
```

```
3
1 2
5 7
6 3
```

1.2. Terminar cuando es 0 0

```
1 import sys
2 for line in sys.stdin.readlines():
3     if line=='0 0\n': break
4     print(sum(map(int, line.split())))
```

```
1 2
5 7
6 3
0 0
1 1 <- no se procesa
```

1.3. Enumerar casos hasta EOF

```
1
2 import sys
3 for c, line in enumerate(sys.stdin.readlines(), 1):
4     print("Case %s: %s\n" % (c, sum(map(int, line.split()))))
```

```
1 2
5 7
6 3
— out
Case 1: 3
Case 2: 12
Case 3: 9
```

1.4. Input matriz nxm hasta EOF

```
1
2 import sys
3 lines = sys.stdin.readlines()
4 i= 0
5
6 while i<len(lines):
7     # Leer dimensiones
8     n, m = map(int, lines[i].split())
9     i += 1
10
11     #Leer la matriz de tamaño n x m
12     matrix = [list(map(int, lines[i+j].split())) for j in
13               range(n)]
14     i +=n
15 print(matrix)
```

```
2 3
1 2 3
4 5 6
— out
[[1, 2, 3], [4, 5, 6]]
```

2. Output Format

```
1 # Mostrar 2 decimales
2 price = 45
3 print(f"The price is {price:.2f} dollars")
4 # The price is 45.00 dollars
5
6 # Usar ',' como separador de miles
7 age = 13800000000
8 print(f"The universe is {age:,} years old")
9 # The universe is 13,800,000,000 years old
10
11 # Convertir a porcentaje con 1 decimal (sin decimales usar 0)
12 score = 0.255
13 print(f"You scored {score:.1%}") # You scored 25.5%
14
15 # Convertir un numero a binario
16 num = 5
17 print(f"The binary version of {num} is {num:b}")
18 # The binary version of 5 is 101
```

3. Macros

```
1 def crear_matriz(n, m, valor =0):
2     return [[valor for i in range(m)] for j in range(n)]
3     #j * m + i + 1    Reemplazar por valor para llenar la lista
4                       en orden de 1 a n*m
5
6 n, m = 3, 4
7 matriz= crear_matriz(n, m, 0)
8
9 for fila in matriz:
10     print(*fila)
11
12 """
13 0 0 0 0
14 0 0 0 0
15 0 0 0 0
16 """
```

4. Estructuras de Datos

Esta sección contiene las estructuras de datos más útiles de Python para programación competitiva.

4.1. Collections - Deque

```
1 from collections import deque
2
3 dq = deque([1, 2, 3])
4 dq.append(4)          # [1, 2, 3, 4]
5 dq.appendleft(0)      # [0, 1, 2, 3, 4]
6 dq.popleft()          # -> 0
```

4.2. Heapq - Priority Queue

```
1 import heapq
2 # Sirve cuando necesitas sacar todo el tiempo el menor o mayor
3   elemento
4 h = []
5 heapq.heappush(h, 3) # Insertar O(log n)
6 heapq.heappush(h, 1)
7 heapq.heappush(h, 5)
8 print(heapq.heappop(h)) # 1 (sacar el menor elemento) O(log n)
```

```

8 print(heapq.nlargest(2, h)) #Lista con los n elementos mas
   grandes (nsmallest para los menores)
9 heapq.merge(h1, h2) #Combina dos listas ordenadas devuelve un
   Iterador (se puede guardar como lista con list())
10
11 # Max heap:
12 h = [-3, 1, -5] #Invertir los numeros al insertar
13 heapq.heapify(h) #O(n)
14 print(-heapq.heappop(h)) # 5 (mayor)

```

4.3. DefaultDict - Grafos

```

1 # Como un dict que crea automaticamente la llave cuando
   accedes a el
2 from collections import defaultdict
3 #Representar grafos con listas de adyacencia.
4 g = defaultdict(list)
5 n, m = 5, 6 #Nodos, aristas
6 edges = [(1,2), (1,3), (2,4), (3,4), (4,5), (5,1)]
7 for u, v in edges:
8     g[u].append(v)
9     g[v].append(u) # si es no dirigido
10
11 # Otro uso guardar elementos sin que se repitan
12 amigos = defaultdict(set)
13 amigos["Juan"].add("Ana")
14 amigos["Juan"].add("Pedro")
15 amigos["Juan"].add("Ana")
16 # 'Juan': {'Ana', 'Pedro'}

```

4.4. Counter - Frecuencias

```

1 from collections import Counter
2 #Cuenta frecuencias mas rapido
3 cnt = Counter([1,2,2,3,3,3])
4 print(cnt[2]) # 2
5 print(cnt.most_common(1)) # [(3, 3)]
6
7 #Funciona con strings
8 conteo = Counter("hola buenas tardes")
9 print(conteo.most_common(2)) #[('a', 3), (' ', 2)]

```

4.5. Bisect - Búsqueda Binaria

```
1 import bisect
2 #Siempre tiene que ser una lista ordenada
3 arr = [1, 3, 3, 5, 8]
4 # Busca en O(log(n))
5 L = bisect.bisect_left(arr, 1) # 0
6 R = bisect.bisect_right(arr, 3) # 3
7
8 #Cuantos elemento hay entre 1 y 3
9 print(R - L) # 3
10 #Insertar en orden
11 bisect.insort(arr, 4) #O(n)
12 # Parametros extra: (lo, hi) indice con los limites abajo y
    arriba para buscar
13 # (key) recibe una funcion como criterio de comparacion ej:
    key=lambda p: p[1] aqui busca el segundo elemento de cada
    lista
```

4.6. Sets - Conjuntos y operaciones en O(1)

```
1 # Crear
2 s = set() # Set vacio
3 s = {1, 2, 3} # Set con elementos
4 # Operaciones basicas (O(1) promedio)
5 s.add(4) # Agregar
6 s.remove(4) # Eliminar (KeyError si no existe)
7 s.discard(4) # Eliminar sin error
8
9 # Operaciones entre conjuntos
10 a | b # Union
11 a & b # Interseccion
12 a - b # Diferencia
13 a ^ b # Diferencia simetrica
14 # Relaciones
15 a <= b # a es subconjunto de b
16 a >= b # a es superconjunto de b
17
18 # Muy usado en BFS/DFS
19 visitados = set()
20 if nodo not in visitados:
21     visitados.add(nodo)
22 # Restriccion
23 # Solo admite elementos hashables (int, str, tuple...), No
    admite list, dict, set, etc.
```

4.7. Matrices

```
1 class matrix(): #Todas las operaciones tienen que ser con otro
    objeto de la misma clase
2     def __init__(self, values:list):
3         self.values = values
4         self.rows = len(values)
5         self.columns = len(values[0])
6
7
8     def __add__(self, other):
9         if self.rows != other.rows or self.columns !=
            other.columns:
10             raise ValueError("Las columnas no son del mismo
                tamano")
11
12         salida = [[0 for _ in range(self.columns)]
13                   for _ in range(self.rows)]
14         for i in range(self.rows):
15             for j in range(self.columns):
16                 salida[i][j] = self.values[i][j] +
                    other.values[i][j]
17
18         return matrix(salida)
19
20
21     def __mul__(self, other):
22         if self.columns != other.rows:
23             raise ValueError("El numero de columnas de la
                primera matriz debe ser igual al numero de
                filas de la segunda.")
24
25         salida = [[0 for _ in range(other.columns)]
26                   for _ in range(self.rows)]
27         for i in range(self.rows):
28             for j in range(other.columns):
29                 for k in range(self.columns):
30                     salida[i][j] += self.values[i][k] *
                        other.values[k][j]
31
32         return matrix(salida)
33
34
```



```

35 def __pow__(self, exponent): #binary exponentiation
36     if self.rows != self.columns:
37         raise ValueError("Solo las matrices cuadradas
38                             pueden elevarse a una potencia.")
39
40     result = matrix([[1 if i == j else 0 for j in
41                       range(self.rows)]
42                      for i in range(self.rows)])
43     base = self
44
45     while exponent > 0:
46         if exponent % 2 == 1:
47             result = result * base
48
49             base = base * base
50             exponent //= 2
51
52     return result
53
54 def __str__(self):
55     return "\n".join(" ".join(map(str, fila)) for fila in
56                       self.values)
57
58 def __eq__(self, other): # Si se necesita comparar
59     return self.values == other.values

```

5. Algoritmos

5.1. Two Sum - Doble puntero

```

1 #La funcion intenta determinar si en el arreglo a (de tamaño
2   n) existen dos números cuya suma sea igual a k.
3 def isPairSum(a, n, k):
4     global x, y
5     i, j = 0, n - 1
6     while i < j:
7         if a[i] + a[j] == k:
8             x, y = i, j
9             return 1
10        elif a[i] + a[j] > k:
11            j = j - 1
12        else:
13            i = i + 1
14    return 0

```

5.2. Sliding Window - Suma Mínima

```
1 # Halla la suma minima de un subarreglo de k elementos
   consecutivos en un array, y devuelve los indices donde
   empieza y termina dicho subarreglo.
2 def window(n:int, k:int, arr:list):
3
4     cursum = res_ind = 0
5     cursum = sum(x for x in arr[:k])
6     minsum = cursum
7
8     for i in range(k, n):
9         cursum = cursum + arr[i] - arr[i - k] #Deslizar ventana
10        if cursum < minsum:
11            minsum = cursum
12            res_ind = i - k + 1
13    return (res_ind, res_ind + k - 1)
14
15 n, k = map(int, input().split())
16 arr = list(map(int, input().split()))
17
18 print(*window(n, k, arr))
19
20 '''
21 #----INPUT----
22 7 3
23 3 7 90 20 10 50 40
24 #----OUTPUT----
25 3 5
26 '''
```

6. Grafos

6.1. Dijkstra - Camino menos costoso

Complejidad: $O((N + M) \log N)$ con heapq.

```
1 from heapq import heappush, heappop
2 import math
3
4 def dijkstra(graph, start, end):
5     dist = {u: math.inf for u in graph}
6     parent = {start: None} #Borrar si no se necesita el camino
7
8     dist[start] = 0
```

```

9     pq = [(0, start)]
10
11     while pq:
12         d, u = heappop(pq)
13
14         if d > dist[u]:
15             continue
16
17         if u == end:
18             break
19
20         for v, w in graph[u]:
21             if dist[v] > d + w:
22                 dist[v] = d + w
23                 parent[v] = u # Guardar de donde viene
24                 heappush(pq, (dist[v], v))
25
26     # Si no hay camino
27     if dist[end] == math.inf:
28         return math.inf, []
29     # Reconstruir camino
30     camino = []
31     actual = end
32
33     while actual is not None:
34         camino.append(actual)
35         actual = parent[actual]
36
37     camino.reverse()
38
39     return dist[end], camino
40
41 #Ejemplo
42 graph = {
43     1: [(2, 4), (3, 1)],
44     2: [(3, 2), (4, 5), (1, 5)],
45     3: [(4, 8), (2, 3)],
46     4: [(3, 2)]
47 }
48 print(dijkstra(graph, 1, 4)) #9 [1, 3, 4]

```

6.2. BFS - Camino más corto

```

1 from collections import deque
2

```

```

3 def bfs(graph, start, end):
4     q = deque([start])
5     parent = {start: None}
6
7     while q:
8         u = q.popleft()
9
10        if u == end:
11            break
12
13        for v in graph[u]:
14            if v not in parent: # parent sirve como visitados
15                parent[v] = u
16                q.append(v)
17        if end not in parent:
18            return []
19        path = []
20        while end is not None:
21            path.append(end)
22            end = parent[end]
23
24        return path[::-1]

```

6.3. BFS - Para obtener distancias/profundidad

```

1 from collections import deque
2
3 def bfs(graph, start):
4     dist = {start: 0}
5     queue = deque([start])
6
7     while queue:
8         node = queue.popleft()
9
10        for neighbor in graph[node]:
11            if neighbor not in dist:
12                dist[neighbor] = dist[node] + 1
13                queue.append(neighbor)
14
15        return dist

```

6.4. DFS - Recursivo

Guarda el recorrido en posorder (variable declarada afuera no retorna nada).
Profundidad maxima de recursion < 1000

```
1 def dfs(graph:dict, node:int, visited:set, orden:list):
2     visited.add(node)
3     for neighbor in graph[node]:
4         if neighbor not in visited:
5             dfs(graph, neighbor, visited, orden)
6     orden.append(node)
```

6.5. DFS - Iterativo

No guarda el camino, solo los nodos que se visitaron (se puede hacer que retorne el orden de visita en preorder). No tiene límite de recursion

```
1 def dfs(graph, start):
2     visited = set()
3     stack = [start]
4
5     while stack:
6         node = stack.pop()
7
8         if node in visited:
9             continue
10
11        visited.add(node)
12
13        for neighbor in reversed(graph[node]):
14            if neighbor not in visited:
15                stack.append(neighbor)
16
17    return visited
```